

END-TERM PROJECT-02 REPORT

ON

FLAPPY BIRD GAME

B.Tech. [Computer Science and Engineering]

Semester: 6th

Section: 23412J2

Course Title & Code: (CSE)

Submitted By: Gatikrushna Bhuyan

Registration No: 2341002188

Guided By:Saurav Sir

Academic Year: 2023-2027



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING AND TECHNOLOGY, INSTITUTE OF
TECHNICAL EDUCATION AND RESEARCH

SIKSHA 'O' ANUSANDHAN (DEEMED TO BE) UNIVERSITY

BHUBANESWAR, ODISHA, INDIA

ABSTRACT

This project focuses on the development of a Flappy Bird-style 2D game using C++ and SFML (Simple and Fast Multimedia Library). The game involves controlling a bird that continuously moves forward while avoiding obstacles (pipes).

The player presses a key or mouse click to make the bird flap and stay airborne. Gravity continuously pulls the bird downward, making the gameplay challenging.

The project demonstrates key concepts such as:

- Object-Oriented Programming (OOP)
- Real-time game loop
- Physics simulation (gravity & velocity)
- Collision detection
- Score and high-score system

Submitted By

Guided By

TABLE OF CONTENTS

Section No.	Topic	Page No.
01	Introduction	1
02	Problem Statement	1
03	Objectives of the Project	2
04	Tools and Technologies Used	3
05	Methodology	4-5
06	Results Analysis and Screenshots	6-7
07	Logical Understanding	8
08	Conclusion	9
09	Future Scopes and Application	10
	References	11
	Annexure	12-17

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Initial Game Screen Showing Bird and Pipes	6
Figure 2	Gameplay Screen with Bird Passing Through Pipes and Score Display	7
Figure 3	Game Over Screen After Collision Detection	7

01. Introduction

Game development is one of the most engaging applications of programming, combining logic, mathematics, graphics, and real-time user interaction. With the advancement of computer graphics and multimedia libraries, it has become easier to design interactive applications and games using high-level programming languages such as C++.

Game development combines programming logic, graphics, and user interaction. This project implements a **Flappy Bird clone** where the player controls a bird navigating through pipes.

The game environment includes:

- A continuously scrolling background
- Dynamic obstacle generation
- Real-time physics-based movement

The goal is to survive as long as possible and achieve a high score.

02. Problem Statement

The problem addressed in this project is to design and develop a **2D Flappy Bird game** that simulates a real-time environment where the player controls a bird and must navigate through continuously incoming pipes without colliding.

Design and develop a **2D Flappy Bird game** that includes:

- Smooth bird movement using physics
- Pipe generation with gaps
- Collision detection system
- Score tracking and high-score saving
- Real-time input handling

Challenges include:

- Gravity simulation
- Accurate collision detection
- Managing object spawning
- Maintaining performance

The goal is to create a smooth, responsive, and engaging gameplay experience that challenges the player's timing and coordination skills.

03. Objectives of the Project

The objectives of this project are defined to provide a clear direction for the design and development of the **Flappy Bird game using C++ and SFML**. The objectives of this practical project are:

- To develop a working 2D Flappy Bird game using the C++ programming language and SFML library.
- To understand and apply Object-Oriented Programming concepts such as classes and modular design.
- To implement real-time user input handling using keyboard controls.
- To design and implement collision detection between game objects.
- To develop a scoring system based on player survival time.
- To implement dynamic difficulty by increasing speed and obstacle frequency.

04. Tools and Technologies Used

The following tools and technologies were used in this project:

Language: C++

Graphics Library: SFML

Compiler: GCC / MinGW

IDE: VS Code / Code Blocks

OS: Windows / Linux

05. Methodology

- Understanding the Problem: Identifying the game requirements such as bird control, pipe generation, collision detection, scoring system, and increasing difficulty.
- System Design: Designing the structure of the game using Object-Oriented Programming concepts with classes like Bird, PipeSystem, HighScore, and Game.
- Algorithm Design: Creating a step-by-step logical solution for game flow, including bird movement, pipe spawning, collision detection, and scoring.
- Game Loop Implementation: Developing a continuous loop that handles input, updates game objects, checks collisions, and renders graphics in real-time.
- User Input Handling: Implementing keyboard or mouse controls (Space key or click) to allow the bird to flap and maintain height.
- Physics Implementation: Applying gravity and upward impulse to simulate realistic bird movement using velocity and acceleration.
- Pipe Generation: Generating pipes dynamically at random vertical positions with gaps to create challenging and unpredictable gameplay.
- Movement Logic: Updating positions of the bird and pipes based on velocity and time (delta time) to ensure smooth motion.
- Collision Detection: Checking for intersection between the bird and pipes, ground, or screen boundaries using bounding box (hitbox) technique and ending the game on collision.
- Score System: Increasing the score when the bird successfully passes through pipes and displaying it on the screen.
- High Score System: Saving and loading the highest score using file handling techniques.
- Difficulty Adjustment: Gradually increasing game difficulty by increasing pipe speed and reducing spawn intervals.
- Graphics Rendering: Drawing the bird, pipes, background, and score using the SFML graphics library.
- Memory Management: Removing pipes that move off-screen to optimize performance and memory usage.

Program Development: Writing the code in C++ using the SFML library to implement physics, pipe movement, collision detection, and scoring system.

- Compilation and Execution: Compiling the program using a suitable compiler (MinGW/GCC) and executing the game to verify functionality and performance.
- Testing and Debugging: Identifying and fixing logical, runtime, and graphical errors to ensure smooth gameplay.
- Performance Optimization: Improving efficiency by reducing unnecessary computations and maintaining stable frame rates using fixed time steps.
- Final Output: Ensuring the game runs correctly with smooth controls, realistic physics, proper visuals, scoring system, and increasing difficulty.

06. Results Analysis and Screenshots

The game was successfully implemented and tested. The player can control the bird using keyboard inputs.

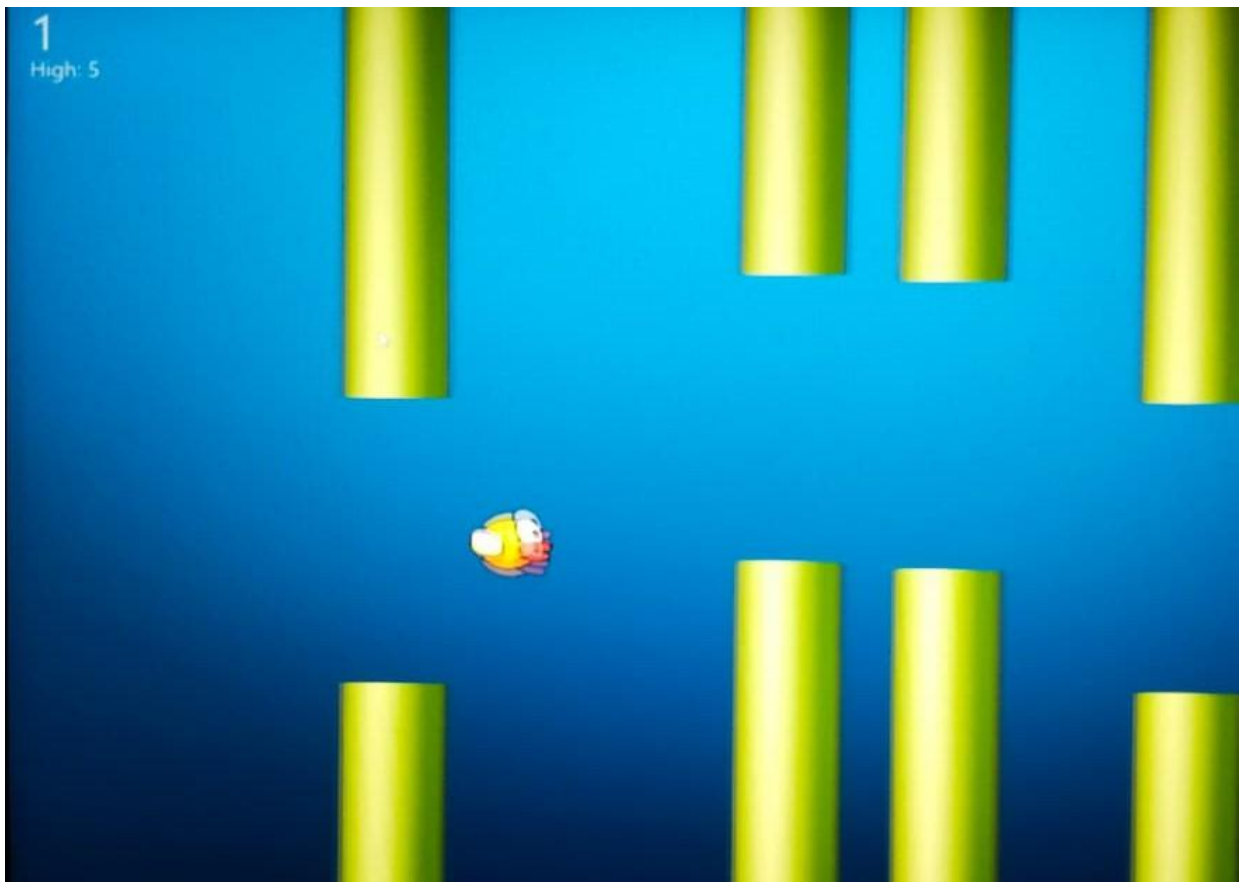
Key observations:

- Smooth gameplay
- Responsive controls
- Accurate collision detection
- Real-time score updates

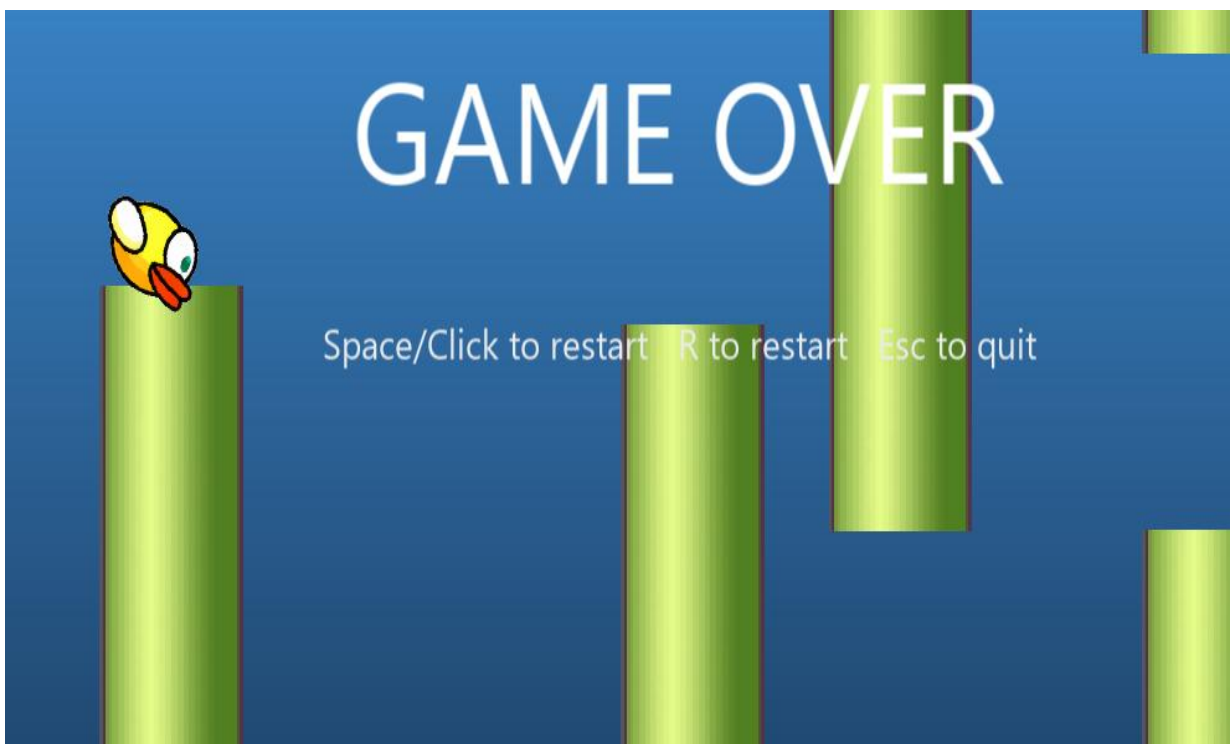
GAME STARTS



BIRD MOVING



GAME ENDS



07. Logical Understanding

The logic of the program is based on:

Bird Movement

- Uses velocity (m_vy)
- Gravity increases downward speed
- Flap applies upward impulse

Collision Logic

- Uses hitbox() function
- Checks overlap with pipes and ground

Game State

- Running
- GameOver

Pipe System

- Random gap generation
- Movement from right → left

08. Conclusion

The project was successfully completed using the C++ programming language and SFML library. It strengthened the understanding of important concepts such as object-oriented programming, real-time input handling, physics-based movement, collision detection, and graphical rendering.

The project provided practical experience in developing an interactive game application and improved problem-solving, debugging, and logical thinking skills. It also demonstrated how programming concepts can be applied to create real-time systems involving continuous motion and user interaction.

Overall, this project enhanced programming confidence and provided a strong foundation for developing more advanced graphical applications and games, especially those involving physics simulation and real-time gameplay mechanics.

09. Future Scopes and Application

The future improvements and applications of this project include:

- Adding advanced features such as multiple levels, power-ups, and different difficulty modes
- Improving graphics by using high-quality textures, animations, and smoother transitions
- Adding sound effects and background music to enhance the gaming experience
- Implementing a main menu, pause, resume, and restart functionality
- Enhancing the high-score system with persistent storage and leaderboard features
- Increasing game complexity by introducing moving pipes or dynamic obstacles

This project can be extended to real-world applications such as game development, simulation systems, and interactive software. It provides a strong foundation for developing more complex graphical applications using C++.

References

- (1) B.M. Harwani, *Practical C Programming*, Packt Publishing
- (2) SFML Official Documentation, Available at: <https://www.sfml-dev.org>
- (3) SFML Tutorials, Available at: <https://www.sfml-dev.org/tutorials/>

Annexure

BIRD.cpp

```
#include "Bird.hpp"

#include <algorithm>

Bird::Bird(const sf::Texture& tex, sf::Vector2f pos, float scale)
    : m_sprite(tex), m_startPos(pos){
    auto s = tex.getSize();
    m_sprite.setOrigin(s.x / 2.f, s.y / 2.f);
    m_sprite.setScale(scale, scale);
    m_sprite.setPosition(pos);
    m_half = {s.x * 0.5f * scale, s.y * 0.5f * scale};}

void Bird::reset(sf::Vector2f pos){
    m_vy = 0.f;
    m_lastRot = 0.f;
    m_sprite.setRotation(sf::degrees(0));
    m_sprite.setPosition(pos);}

void Bird::flap(float impulse){
    m_vy = impulse;}

void Bird::update(float dt, float gravity, float maxFall){
    m_vy = std::min(m_vy + gravity * dt, maxFall);
    m_sprite.move(0.f, m_vy * dt);}

void Bird::setRotationFromVelocity(float factor, float minDeg, float maxDeg){
    float target = std::clamp(m_vy * factor, minDeg, maxDeg);
    m_lastRot += (target - m_lastRot) * 0.18f;
    m_sprite.setRotation(sf::degrees(m_lastRot));}

sf::FloatRect Bird::hitbox() const{
    auto p = m_sprite.getPosition();
    float w = m_half.x * 2, h = m_half.y * 2;
    return {
        {p.x - m_half.x + w * 0.15f, p.y - m_half.y + h * 0.15f},
        {w * 0.7f, h * 0.7f} }; }

void Bird::draw(sf::RenderTarget& target) const{
    target.draw(m_sprite); }
```

Explanation:

- Represents the player-controlled bird in the game
- Stores bird properties such as sprite, position, velocity, and rotation
- Initializes the bird with texture, scaling, and starting position
- Simulates movement using gravity and upward impulse (flap)
- Updates bird position smoothly using velocity and time (delta time)

PIPESYSTEM.cpp

```
#include <SFML/Graphics.hpp>

#include <vector>

#include <random>

class PipeSystem {

public:

    PipeSystem() = default;

    PipeSystem(const sf::Texture& tex, float width);

    void reset(float winW, float groundY);

    void update(float dt, float winW, float groundY, int score);

    void draw(sf::RenderTarget& target) const;

    bool collides(const sf::FloatRect& bird, float groundY) const;

    int getScore(float birdX);

private:

    struct Pipe {

        float x{}, gapY{}, gap{};

        bool active{}, passed{}; };

    void spawn(float winW, float groundY);

    void recycle(float pipeW);

    const sf::Texture* m_tex{};

    float m_width{};

    std::vector<Pipe> m_pipes;

    std::mt19937 m_rng{std::random_device{}() }; }
```

Explanation:

- Manages all pipe obstacles in the game
- Stores pipe properties such as position, gap size, and active state
- Generates pipes dynamically with random gap positions for unpredictable gameplay
- Updates pipe positions continuously to create scrolling movement
- Detects collisions between the bird and pipes using bounding box logic
- Tracks when the bird passes pipes to update the score

HIGHSCORE.cpp

```
#include "HighScore.hpp"

#include <fstream>

HighScore::HighScore(std::string path) : m_path(std::move(path)){

int HighScore::load(){

    std::ifstream f(m_path);

    int v = 0;

    if (f.good()) f >> v;

    m_value = v;

    return m_value; }

void HighScore::saveIfHigher(int score){

    if (score <= m_value) return;

    m_value = score;

    std::ofstream f(m_path, std::ios::trunc);

    if (f.good()) f << m_value; }

int HighScore::value() const{

    return m_value; }
```

Explanation:

- Manages the storage and retrieval of the game's high score
- Stores the file path where the high score is saved
- Loads the high score from a file when the game starts
- Updates the high score only if the current score is higher

MAIN.cpp

```
#include "Game.hpp"

int main(){

    Game g;

    return g.run(); }
```

Explanation:

- Entry point of the program
- Creates an object of the Game class
- Calls the run() function to start the game loop
- Controls the execution flow of the entire game
- Keeps the program running until the game window is closed